

Word Frequency — Estudo de Caso

Um projeto prático construído do zero com o Claude Code: front-end vanilla JS, backend Node.js/Express e testes automatizados, em seis fases documentadas.

Consolidado a partir de App.md, CLAUDE.md e Word-Frequency-Relatorio.pdf

- Tier 1 · Iniciante
- HTML · CSS · JS
- Node.js + Express
- + Landing Page

O que é o Word Frequency

Aplicação classificada como Tier 1 (Iniciante), cujo objetivo é calcular a frequência de palavras em um bloco de texto. O usuário informa um texto — digitado ou colado, com até 2048 caracteres — clica no botão Translate e recebe uma tabela com cada palavra única encontrada e o número de vezes que ela aparece, ordenada da mais para a menos frequente. É uma introdução prática ao modelo Bag of Words, técnica usada em busca, ordenação e análise semântica de texto.

ESPECIFICAÇÃO	GUIA DE DESENVOLVIMENTO
App.md — histórias de usuário (Tier 1)	CLAUDE.md — stack, fases e convenções
STATUS	PASTA DO PROJETO
6 fases concluídas + 2 bônus + landing page	World_free/Primeiro Projeto/

Funcionalidades obrigatórias

- Caixa de texto, botão "Translate" e tabela de frequência visíveis na tela.
- Entrada de texto grande, com limite de 2048 caracteres (contador visual).
- Mensagem de erro quando o campo de texto está vazio.
- Tabela populada com palavra e contagem de ocorrências, em ordem decrescente de frequência.

Recursos bônus entregues

- Gráfico de barras (column chart) com Chart.js, limitado às 15 palavras mais frequentes.
- Análise de texto extraído de uma URL informada pelo usuário, via backend Express dedicado (evita CORS).

STACK

Stack técnica

Tecnologias observadas na estrutura real do projeto, coerentes com o que o CLAUDE.md e o relatório documentam.

CAMADA	TECNOLOGIA	MOTIVO
Front-end	HTML + CSS + JavaScript vanilla	Aplicação simples, sem estado complexo — dispensa framework e build step.
Gráfico	Chart.js (via CDN)	Biblioteca leve, cobre column chart sem exigir bundler.
Backend	Node.js + Express	Necessário só para buscar conteúdo de URLs externas, evitando CORS.
Extração de HTML	cheerio	Parsing e remoção de tags irrelevantes para extrair texto limpo.
Testes	node:test (nativo)	Runner embutido no Node — zero dependências extras.
Persistência	Nenhuma	Toda a contagem roda em memória, no cliente.

Fases de desenvolvimento

1 Estrutura estática Layout inicial: textarea, botão "Translate" e tabela vazia; limite de 2048 caracteres.	CONCLUÍDA
2 Lógica de contagem (core) Tokenização, contagem em função pura (sem tocar no DOM), renderização inicial sem ordenação.	CONCLUÍDA
3 Ordenação e validações Ordenação decrescente por frequência, mensagem de erro para campo vazio, tratamento de edge cases.	CONCLUÍDA
4 Polimento de UX Responsividade, feedback visual, tema claro/escuro automático, acessibilidade (atributos ARIA).	CONCLUÍDA
5 Bônus: gráfico Column chart com Chart.js; cores acompanham o tema claro/escuro da aplicação.	CONCLUÍDA
6 Bônus: análise via URL Alternância texto/URL, backend com endpoint /api/extract, tratamento de erros de rede.	CONCLUÍDA

Arquitetura e fluxo de dados

A separação de responsabilidades é o princípio central: a lógica de contagem de palavras (js/wordFrequency.js) é pura — recebe texto e devolve dados, sem depender do navegador — o que a torna simples de testar isoladamente. A manipulação da interface fica isolada em js/main.js, que apenas consome essas funções puras e repassa o resultado para js/chart.js.

- main.js lê o texto (digitado ou via URL) → chama countWordFrequency() e sortByFrequency() de wordFrequency.js.
- O resultado alimenta a tabela e renderChart() ao mesmo tempo — nenhuma função de cálculo conhece o DOM.
- server/index.js é um serviço Express separado, com seu próprio package.json, usado só no modo URL.
- landing.html não participa desse fluxo — é a porta de entrada explicativa que leva até index.html.

Testes automatizados: a lógica de tokenização, contagem, ordenação e validação de feedback é coberta por testes executados com o runner nativo do Node.js (node:test), sem dependências externas.

SUÍTE tests/wordFrequency.test.js + tests/feedback.test.js	EXECUÇÃO npm test → node --test
RESULTADO (CHECADO NESTA SESSÃO) 15 de 15 testes aprovados	ESCOPO COBERTO tokenize, countWordFrequency, sortByFrequency, validateFeedback

O relatório original (31/ago/2026) registra 8 testes só de wordFrequency.js; o projeto evoluiu desde então com um recurso de feedback do usuário (js/feedback.js), hoje também coberto por testes.

Landing page: página estática adicionada posteriormente (landing.html) para apresentar a aplicação antes de o visitante abrir a ferramenta: hero com chamada para ação, passo a passo de uso, grade de funcionalidades, stack técnica e um CTA final — todos levando até index.html. Reaproveita os tokens de cor do style.css do app, sem introduzir lógica testável nova.

O que essa aula ensinou

- Construir um projeto completo com o Claude Code, da spec (App.md) ao código funcional e testado — não uma aula isolada, mas um fluxo real de desenvolvimento.
- Traduzir uma especificação em histórias de usuário (App.md) para um guia de execução prático (CLAUDE.md), com stack, fases e convenções de código definidas antes de codar.
- Trabalhar em fases incrementais — estrutura, lógica pura, validações, UX, e só então os bônus — em vez de tentar tudo de uma vez.
- Separar lógica pura (testável, sem DOM) da camada de interface, permitindo cobertura de testes sólida com o runner nativo do Node (node:test), sem dependências extras.
- Estender o escopo original com um backend dedicado (Node.js + Express) só onde estritamente necessário (contornar CORS), mantendo o resto do app 100% client-side.
- Fechar o ciclo com uma landing page e refinamentos de UX (acessibilidade, tema claro/escuro, feedback do usuário) que não estavam no escopo obrigatório, mas elevam o resultado final.

CONSIDERAÇÕES FINAIS

O projeto está com todas as histórias de usuário obrigatórias e os dois recursos bônus da especificação implementados e validados. A aplicação roda inteiramente no navegador para o fluxo principal, com um backend leve e opcional apenas para a análise via URL.

Possíveis evoluções fora do escopo original:

- Deploy do front-end (ex.: GitHub Pages) e do backend (ex.: Render, Railway).
- Lista de stop words configurável, para ignorar palavras muito comuns.
- Exportação da tabela de frequência em CSV.
- Histórico de análises anteriores (exigiria persistência).

Fontes: World_free/Primeiro Projeto/App.md, CLAUDE.md e Word-Frequency-Relatorio.pdf (31 de agosto de 2026), além da estrutura real de arquivos do projeto conferida nesta sessão.